СОД Stream-CAD · Аудит 6.0

Точечный аудит воспроизводимой точки `59faf20` («4D») — вьювер, фильтры, хедер. С готовым и проверенным хотфиксом.

Параметр	Значение
Точка аудита	`59faf20` — «4D: интеграция Аудита 5.0 (Спринты 7–8)»
origin/main	`fe2af42` («5C») — на 3 коммита впереди точки аудита
Запрос владельца	починить вьювер, сжать фильтры, проверить кнопки хедера; подготовить вводные кодеру; вернуть задеплоенное
Запреты	tokens.json · структура таблицы · services.py · pdf_viewer.js (кроме точечных правок)
Проверка	полный набор тестов на `59faf20` + хотфикс: **403 / 403 OK**; замеры Playwright прилагаются
Результат	`sod_hotfix_6.patch` — 5 файлов, применяется на чистую точку без конфликтов

3 / 3
поломки подтверждены, у каждой измеренная причина

413 → 18 px
дрейф основного листа во вьювере: было → стало

128 → 53 px
высота фильтр-панели: было → стало

Все три симптома воспроизведены на поднятой копии точки `59faf20` с живой базой и замерены инструментально (Playwright, вьюпорт 1440×860 и 1366×860). Для двух из трёх подготовлен готовый хотфикс; третья (хедер) на чистой точке не воспроизводится — раздел 3.3 объясняет, что владелец видит на деплое и как это чинится.

1. Вводные (зафиксированы в формулировке владельца)

Заказчик зафиксировал рамки работ до начала аудита — это снимает двусмысленности и defines критерии приёмки. Ниже дословная фиксация вводных; они же продублированы в файле `README_TASK.md` из пакета, который передаётся исполнителю.

Категория	Содержание
Текущая точка	`git reset --hard 59faf20` (коммит «4D») — от неё ведётся работа
Сломано 1	PDF-вьювер: основной лист статичен вверху, полоса миниатюр прокручивается отдельно — рассинхрон
Сломано 2	Фильтры реестра: в две строки, занимают много места
Сломано 3	Цвет кнопок в хедере — «тёмный текст на тёмном фоне»
Нужно	1) синхронизация листа и миниатюр; 2) фильтры в одну строку; 3) корректные цвета кнопок хедера
Нельзя	менять `tokens.json`; менять структуру таблицы документов; трогать `services.py`; трогать `pdf_viewer.js` (кроме точечных правок)

Категория	Содержание
Отдельный вопрос	как вернуть состояние, которое сейчас задеплоено, — в GitHub уже уехали другие коммиты (раздел 6)

2. Что фактически находится в репозитории

Первый шаг — сверить картину в голове владельца с реальной историей. Ключевой факт: фронтенд-файлы точки «4D» (стили, скрипты, шаблоны) байт-в-байт совпадают с эталоном Спринтов 7–8 из Аудита 5.0 — сверены все 25 фронтенд-файлов эталонного набора. Расходятся только 4 бэкенд-файла (`settings.py`, `services.py`, `views.py`, сгенерированный `sw-bypass-list.js`) — там живёт интеграция кодера: API Gateway, feedback, демо-команда. Значит, все три симптома — не «кривое слияние», а свойства самой интеграции 4D, и чинить их надо поверх 4D.

Коммит	Метка	Содержание	Отношение к задаче
`92f237c`	—	база Аудита 5.0 (PROGRESS, HANDOFF, контракты Feedback)	история
`6d5c19a…fcf5f96`	1A–2C	404/500, демо-команда, Feedback + hotfix fallback	история
`15306ae`, `dac8814`	4A	API Gateway (ApiToken, Bearer) — **два дубль-коммита**	мусор в истории, не вредоносен
`59faf20`	**4D**	**интеграция Спринтов 7–8** + всё вышеперечисленное	**точка аудита**
`c32f5f3`	5A	SHARE_LINK_TTL из env (30 дней) + тост срока	фича, к трём поломкам не относится
`503089a`	5B	кэш прав, дерево папок, select_related (104→25 запросов)	оптимизация, отдельное решение
`fe2af42`	5C	хедер #F5F7FA, компактный фильтр-бар, фикс вьювера	**кодер уже чинил те же 3 вещи**

Таблица 1 — история origin/main от базы Аудита 5.0 до текущего main

Вывод из таблицы принципиален: в «5C» исполнитель уже пытался закрыть ровно те же три проблемы, о которых говорит владелец. Его вьювер-фикс и компактный фильтр-бар проверены ниже замерами и признаны правильными по механике — хотфикс берёт их за основу. Однако 5C сделан с нарушением новой границы: там изменён `tokens.json` (хедер #FFFFFF → #F5F7FA) и theme-color в двух шаблонах. Эти правки в хотфикс **не включены**. Содержимое 5A/5B/5C рекомендуется сохранить в отдельную ветку — это самостоятельные продуктовые решения, которые владелец примет или отклонит отдельно.

3. Диагностика: три симптома, три причины, замеры

3.1. Вьювер: почему лист и миниатюры рассинхронизированы

Вьювер рендерит по одному листу: `renderPage()` кладёт текущую страницу в контейнер `.stage`, полоса листов — это `#thumbRail` с кнопками-миниатюрами. Рассинхрон собирается из трёх дефектов, действующих одновременно:

- **Дефект A (CSS).** У `.thumb-rail` нет ограничения высоты (`max-height/align-self`). Полоса растёт на высоту всего документа, сама не получает прокрутку, и скролл перехватывает общий предок `.stage`.
- **Дефект B (JS).** `updateThumbsActive()` вызывает `scrollIntoView({block:'nearest'})` на активной миниатюре. Метод прокручивает **все** прокручиваемые предки — включая `.stage`: при смене активной страницы основной лист физически уводится из зоны видимости.
- **Дефект C (JS).** `goToPage()` рендерит новую страницу, но не возвращает `.stage` к верху листа — после перехода пользователь видит пустое полотно, а лист «прилип» выше/ниже.

Воспроизведение на чистом 4D (тестовый PDF из 5 листов, полоса открыта): клик по миниатюре «5» — и `.stage` уезжает на **413 px**, лист оказывается выше видимой зоны (`pageTopInStage = −383`), полоса при этом сама не прокручивается (`railTop = 0`). Пользователь видит ровно описанный симптом: лист статичен/исчез, полоса живёт своей жизнью.

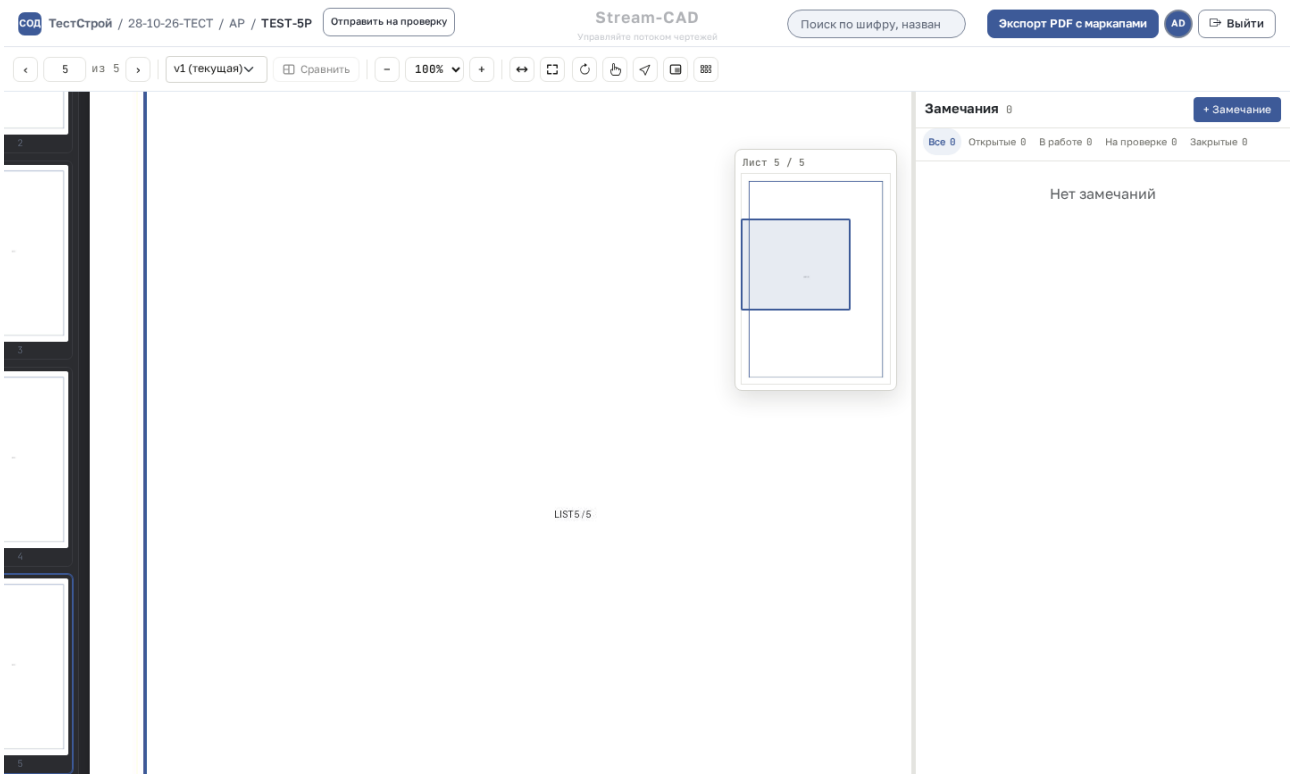


Рис. 1 — Чистый 4D, ДО фикса: выбран лист 5/5, основной лист смещён за экран, видно пустое полотно и полосу, живущую отдельно

Исправление (входит в хотфикс): высота полосы фиксируется (`align-self: stretch; max-height: 100%`), прокрутка миниатюр выполняется вручную только внутри rail (`scrollThumbIntoRail()`), а после

`goToPage()` контейнер явно возвращается к верху листа (`focusViewerPage()`). Изменение `pdf_viewer.js` — **+45/−2 строки, две точечные функции**: в рамках запрета «кроме точечных правок». Контрольный замер после фикса: `stageTop = 18` px (штатный отступ 12 px + граница), `pageTopInStage = 12` px — лист у верхней кромки и виден, `railTop = 437` px — полоса прокрутилась сама и подсветила активную миниатюру.

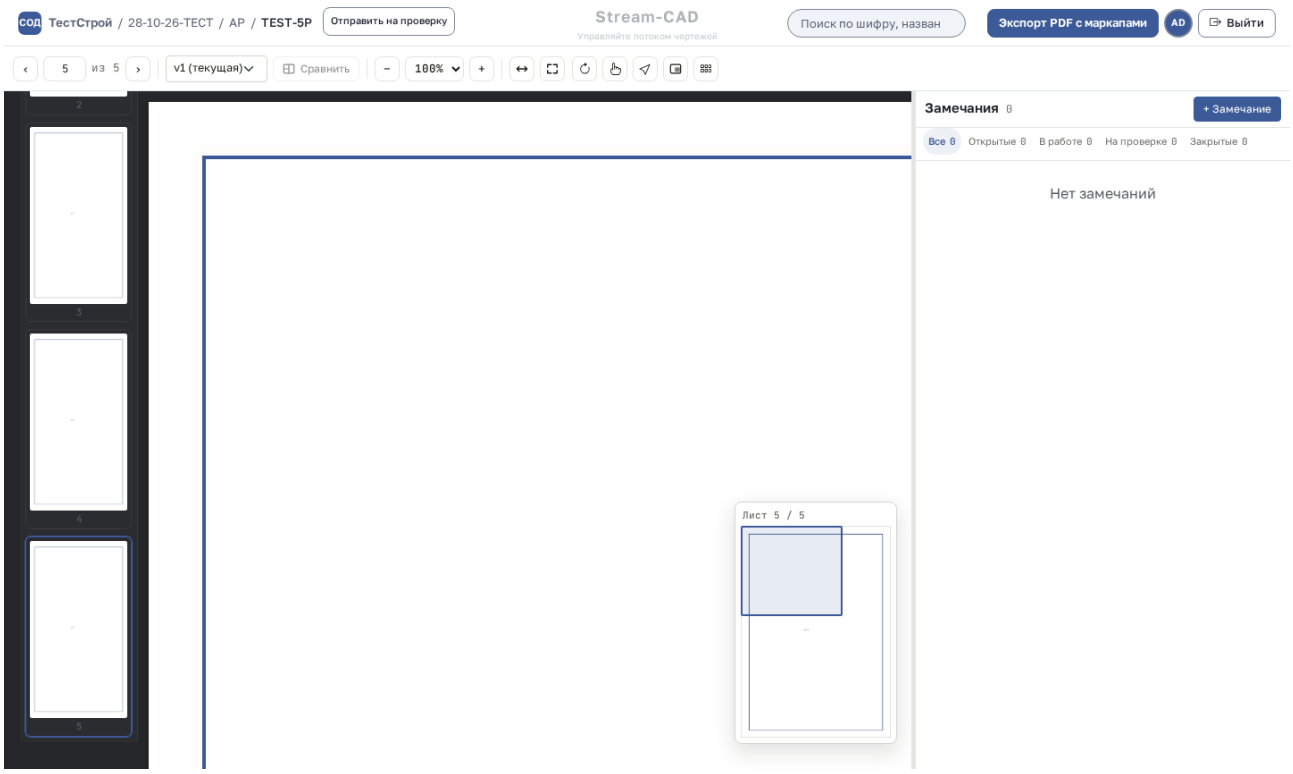


Рис. 2 — После хотфикса: лист 5/5 вверху и виден, полоса прокрутилась к активной миниатюре синхронно

3.2. Фильтры: две строки → одна

Панель фильтров реестра (`.filter-bar`) собрана на flex-обёртке `flex-wrap: wrap` с `min-width: 130px` у пяти полей и отдельным полем поиска шириной от 200 px. На ширинах 1120–1440 px набор гарантированно не помещается в одну строку: поля переносятся, сверху добавляется ряд подписей, снизу — отдельная строка «Поиск». Замер на чистом 4D при 1366 px: **высота панели 128 px, 2 ряда контролов** плюс подписи.

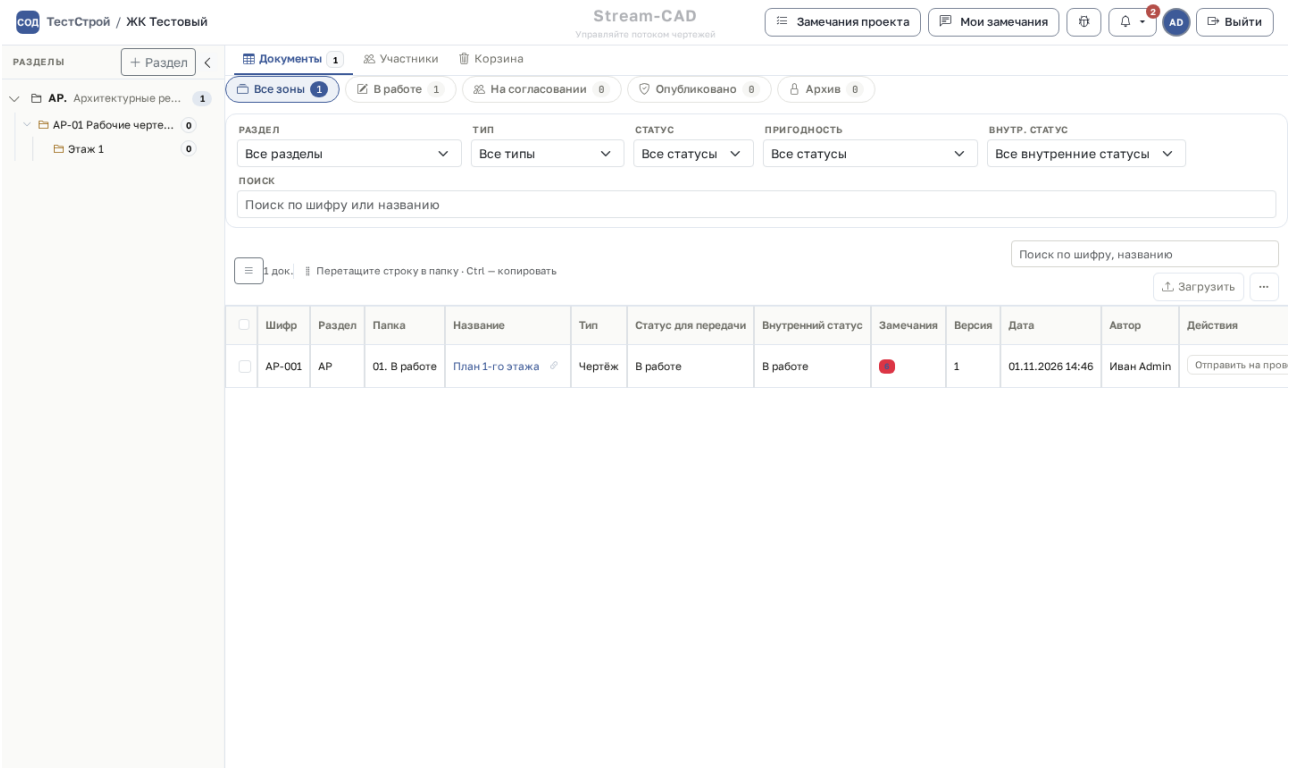


Рис. 3 — Чистый 4D, ДО: панель фильтров в две строки (128 px), вторая строка — отдельное поле поиска

Компактный вариант (входит в хотфикс, заимствован из 5C): перенос запрещён (`flex-wrap: nowrap`), поля получают `flex: 1 1 0` с `min-width: 0`, высота контролов снижена до 27 px, отступы и радиусы ужаты, подписи уменьшены. Контрольный замер после фикса: **53 px** на всех проверенных ширинах 1120 / 1280 / 1366 / 1440 px, горизонтального переполнения нет. Структура таблицы документов не тронута — правка только в `redesign.css`.

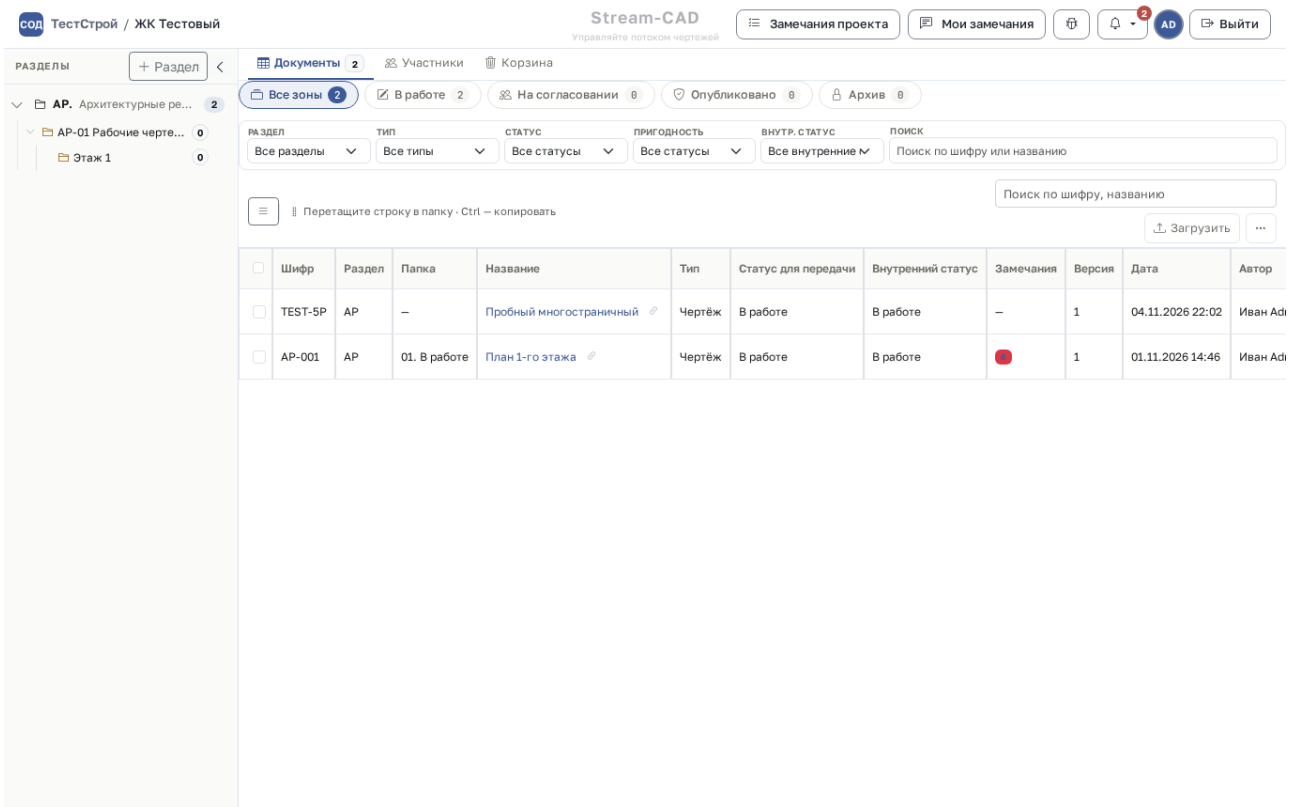


Рис. 4 — После хотфикса: фильтры в одну строку (53 px), поиск в той же линии, переполнения нет

3.3. Хедер: «тёмный текст на тёмном фоне» на чистом 4D не существует

По хедеру проведён инструментальный протокол: на четырёх страницах (список проектов, дашборд, реестр, вьювер) в обеих темах сняты вычисленные цвета и эффективные фоны всех элементов топбара (29 контролов) и посчитан контраст по WCAG. Итог: в светлой теме нарушений контраста нет вообще; в тёмной теме топбар тоже остаётся белым — тёмная тема не переопределяет header-переменные, и нарушений также нет. Проще говоря, на воспроизводимой точке 4D кнопки хедера корректны в обеих темах.

Откуда тогда симптом у владельца? Сочетание «тёмный текст на тёмном фоне» — точная сигнатура **смешанного состояния файлов**: если на сервере лежит старый `tokens.css` (ещё со Спринта 4: графитовый хедер #1F2937) вместе с новым `redesign.css` (Спринт 8: цвет ghost-кнопок берётся из `--c-text-on-dark` = #1F2430, тёмный) — кнопки становятся тёмными на графите. Второй кандидат — сервис-воркер: он отдал закэшированный `tokens.css` вместе со свежим `redesign.css`. Обе причины лечатся не правкой цветов, а приведением деплоя к одному коммиту и принудительным обновлением кэша (раздел 6). Поэтому в хотфиксе — только бампы версий `?v=` в двух шаблонах: они принуждают браузер и SW забрать свежие стили.

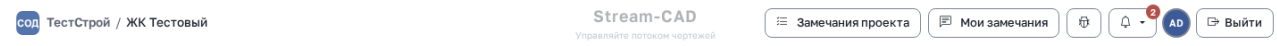


Рис. 5 — Тёмная тема на 4D: топбар остаётся светлым, кнопки читаемы — дефекта «тёмное на тёмном» нет

4. Хотфикс sod_hotfix_6.patch: состав и проверка

Патч подготовлен от чистой точки `59faf20` и проверен тремя способами: `git apply --check` на нетронутом коммите проходит без конфликтов; полный тестовый набор после применения — **403 / 403 OK**; ручные замеры раздела 3 воспроизводят цифры «после». Состав:

Файл	Изменение	Зачем
`css/viewer.css`	+7/–2	высота полосы листов: `align-self: stretch`, `max-height: 100%`; `#pdfViewer` — flex (дефект A)
`js/pdf_viewer.js`	+45/–2	две точечные функции: `scrollThumbIntoRail()` и `focusViewerPage()` (дефекты B, C) — точечные правки в рамках запрета
`css/redesign.css`	+37/–27	компактный фильтр-бар: nowrap, flex-поля, 27 px контролы (симптом 2)
`base.html`	6 строк	только бампы `?v=20261109-h6` — сброс кэша CSS/JS у браузера и сервис-воркера
`document_detail.html`	16 строк	тот же бамп версий для страницы вьювера

Таблица 2 — состав хотфикса (итого +100/–38 строк; пути указаны от static/)

Что осознанно **не включено**: изменение `tokens.json` и `theme-color` на #F5F7FA из 5C (запрещены владельцем — и не нужны: белый хедер корректен); тост срока ссылки из `app.js` (это фича 5A, не про три поломки); бэкенд-оптимизации 5B (`services.py` — запрещён); изменения тестов. Ни один запрет владельца не нарушен: `tokens.json` не тронут, структура таблицы не менялась, `services.py` не тронут,

`pdf_viewer.js` — только две точечные функции.

5. Предписание исполнителю (ТЗ)

Передать исполнителю нужно три вещи: этот отчёт, файл `sod_hotfix_6.patch` и `README_TASK.md` (одностраничная версия вводных). Порядок работ:

- **Шаг 1.** Локально: `git reset --hard 59faf20` (уже сделано владельцем). Убедиться: `git status` чист.
- **Шаг 2.** `git apply --check sod_hotfix_6.patch` — должно пройти молча; затем `git apply sod_hotfix_6.patch`.
- **Шаг 3.** `python manage.py test` — ожидание: **403 теста, все OK** (на чистом 4D проверено).
- **Шаг 4.** Зафиксировать коммитом: `git commit -am "hotfix(Audit6): viewer sync + compact filters"` и отправить в согласованную ветку (раздел 6 определяет, в какую).
- **Шаг 5.** Деплой строго из ОДНОГО коммита (никаких «скопированных поверх» файлов — именно смешанные состояния дают «тёмное на тёмном»), затем принудительное обновление: жёсткая перезагрузка + `unregister`/перерегистрация сервис-воркера, у PWA — переустановка.

Чеклист приёмки (по нему принимает владелец):

- Вьювер: открыть многостраничный PDF, открыть полосу листов, клик «5» — лист вверху и виден, полоса прокрутилась к миниатюре; перелистывание стрелками — лист не уезжает.
- Фильтры: реестр при 1366 px — панель одной строкой, высота ≈53 px; фильтрация и «Сбросить всё» работают.
- Хедер: кнопки читаемы в светлой и тёмной теме на всех страницах; после деплоя контраст-скрипт не даёт нарушений.
- Тесты: 403 OK. Токены: `python scripts/build_design_tokens.py --check` — OK.

Судьба 5A/5B/5C. Не удалять молча: `git branch backup/5x fe2af42` сохраняет работу. Дальше — решение владельца: API Gateway (4A) и TTL ссылок (5A) — самостоятельные фичи; N+1-оптимизация 5B полезна, но трогает запрещённый `services.py` — принимать её можно только отдельной согласованной волной с ревью.

6. Как вернуть задеплоенное: три сценария

Сначала — выяснить, что реально работает на сервере. На машине деплоя в корне проекта: `git log --oneline -1`. Если проект на сервере обновлялся копированием файлов, сверить отпечатки: `git hash-object static/js/pdf_viewer.js static/css/viewer.css static/css/redesign.css` на сервере против `git rev-parse 59faf20:static/js/pdf_viewer.js` (и аналогично) локально. Дальше — один из трёх сценариев.

Сценарий А — рекомендуемый: хотфикс-ветка от 4D, main чистится отдельно

```
git checkout -b hotfix/audit6 59faf20
git apply --check sod_hotfix_6.patch && git apply sod_hotfix_6.patch
git commit -am "hotfix(Audit6): viewer sync + compact filters"
git push -u origin hotfix/audit6
# деплой из hotfix/audit6; main потом чинится revert-ом (сценарий Б) и PR-ом
```

История не переписывается, ничья работа не теряется, прод получает проверенное состояние уже сегодня. Минус — main временно остаётся «навреначенным», порядок наводится вторым шагом.

Сценарий Б — вернуть main к содержимому 4D, сохранив историю

```
git branch backup/5x fe2af42 # сохранить 5A–5C в ветку
git checkout main
git revert --no-commit fe2af42 503089a c32f5f3 # обратный порядок!
git commit -m "revert: откат 5A–5C к состоянию 4D"
git push origin main
```

В main появляется revert-коммит, и содержимое ветки совпадает с 4D. Возможны конфликты, если 5B и 5C трогали одни строки с разных сторон — тогда резолвить в пользу 4D. Затем хотфикс накладывается обычным образом и уходит в main.

Сценарий В — радикальный: переписать main на 4D

```
git branch backup/5x fe2af42 # страховка до любого force!
git checkout main
git reset --hard 59faf20
git push --force-with-lease origin main
```

Допустимо, только если с репозиторием работает один человек и никто не успел клонировать/разветвить 5A–5C. `--force-with-lease` вместо `--force` — обязательная гигиена: не затрёт чужой пуш, произошедший между просмотров и пушем. Во всех сценариях после — тесты, деплой одним коммитом, чеклист раздела 5.

Резюме: обе исправимые поломки имеют измеренные причины и готовый проверенный патч; третья на чистой точке отсутствует и является следствием смешанного деплоя — лечится приведением файлов к одному коммиту. Пакет «Аудит 6.0» содержит отчёт, патч, одностраничные вводные и контрольные скриншоты до/после — этого достаточно, чтобы исполнитель сдал работу с первого раза.